

Part 2 — Operationalising the crawler for billions of URLs

Design documentation for the brief's Part 2: "operationalize the collection of billions of URLs using the code developed", with the required outputs — storage of metadata and content, a unified schema, SLOs and SLAs, and monitoring — and the next steps for cost, reliability, performance and scale. Every number carries a label: **measured** (where and how, with n), **assumed** (why this value) or **derived** (arithmetic shown). Derived numbers are produced by `scripts/ledger.py`, which prints the tables in §2 and §12 from the labelled inputs.

1. The problem in the brief's terms

The service must "take billions of URL and identify the metadata on the page, allow millions of requests on the content, and optimize for cost, performance and availability." The Part 2 input is a "list of billions of URLs send in via a text file and/or in MySQL for a given year month", and the brief's example is "billions of URLs for amazon.com, walmart.com, bestbuy.com etc for July."

That example fixes the shape of the input, and the shape decides which constraint binds:

Regime	Input shape	Binding constraint	What the design must do
Few-domain (the brief's example)	billions of URLs on a handful of hosts	the request rate each host will accept	spend a fixed per-host budget on the most valuable URLs; report completion honestly as a function of that budget
Many-domain	billions of URLs across ~1M hosts	CPU for parsing	size a fleet, keep the queue and stores fed, hold per-host politeness as a fairness rule

The unit of work in both regimes is the Part 1 crawler: one URL in, one `CrawlResult` out, with `status`, `reason`, `robots_state`, `metadata`, `body`, `page_type` and `topics`. Everything below is the machinery that feeds it URLs at the right rate and stores what it returns.

2. Capacity — two regimes

Planning volume is 10 billion URLs per month (assumed: the brief says "billions"; 10e9 is the upper planning case; at 2 billion divide every linear figure by five).

2.1 Many-domain regime

ID	Quantity	Value	Label	Inputs / working
N-00	URLs per month	10e9	assumed	brief: "billions"; upper planning case
N-01	Sustained fetch rate	3,858/s	derived	N-00 / (30 × 86,400)
N-02	2× diurnal peak	7,716/s	derived	2 × N-01; 2× is a planning convention (assumed)

ID	Quantity	Value	Label	Inputs / working
N-03	HTML per page	200 KB	assumed	planning median; the CNN and Amazon fixtures are 5.6 MB and 1.6 MB, large-publisher outliers
N-04	Wire compression	5:1	measured	gzip on both fixtures: 5.0 and 5.2, n=2
N-04b	Wire bytes per page	40 KB	derived	N-03 / N-04
N-05	CPU per page, dev machine	500 ms	measured	parse 461 ms + topics 38.5 ms = 499.5 ms, rounded to 500 for every derivation; n=5 per fixture; Apple M4
N-05b	CPU per page, production core	750 ms	assumed	1.5 × N-05; the first PoC measurement replaces it
N-06	Cores	2,894	derived	N-01 × 0.750 s
N-07	16-vCPU instances at 100% / 70% utilisation	181 / 258	derived	N-06 / 16; / 0.7
N-08	Core-hours per month / per million URLs	2,083,333 / 208	derived	N-06 × 720; / 10,000
N-25	Memory per worker process / per instance	400 MB / 6.4 GB	measured → derived	RSS 352–379 MB after one parse, n=1 each; × 16 processes
N-26	In-flight fetches, total / per instance	7,716 / 43	derived	N-01 × 2 s (assumed mean fetch latency); / 181
N-11b	Raw HTML per month, uncompressed	2.0 PB	derived	N-00 × N-03
N-11	Raw HTML per month, stored compressed	400 TB	derived	N-00 × N-04b
N-10	Metadata record	10 KB	measured → assumed	compact JSON of docs/samples: 9,609 B (Amazon), 6,064 B (CNN); n=2; 10 KB rounds up as the planning value
N-10b	Metadata per month, raw / compressed	100 TB / 25 TB	derived	N-00 × N-10; 4:1 column compression (assumed)
N-28	Input file	800 GB	derived	N-00 × 80 B mean URL (assumed)
N-29	Frontier table, 10e9 rows	1.2 TB	derived	N-00 × 120 B per row (assumed: host, url_hash, url, priority, timestamps)

Sensitivity: N-03 is the least certain input. At 1 MB per page, N-11 becomes 2 PB stored and wire bandwidth 6.2 Gbps; at 200 KB it is 1.2 Gbps. N-05b is the second: at 2.0× instead of 1.5×, N-06 is 3,858 cores and N-07 is 241 instances.

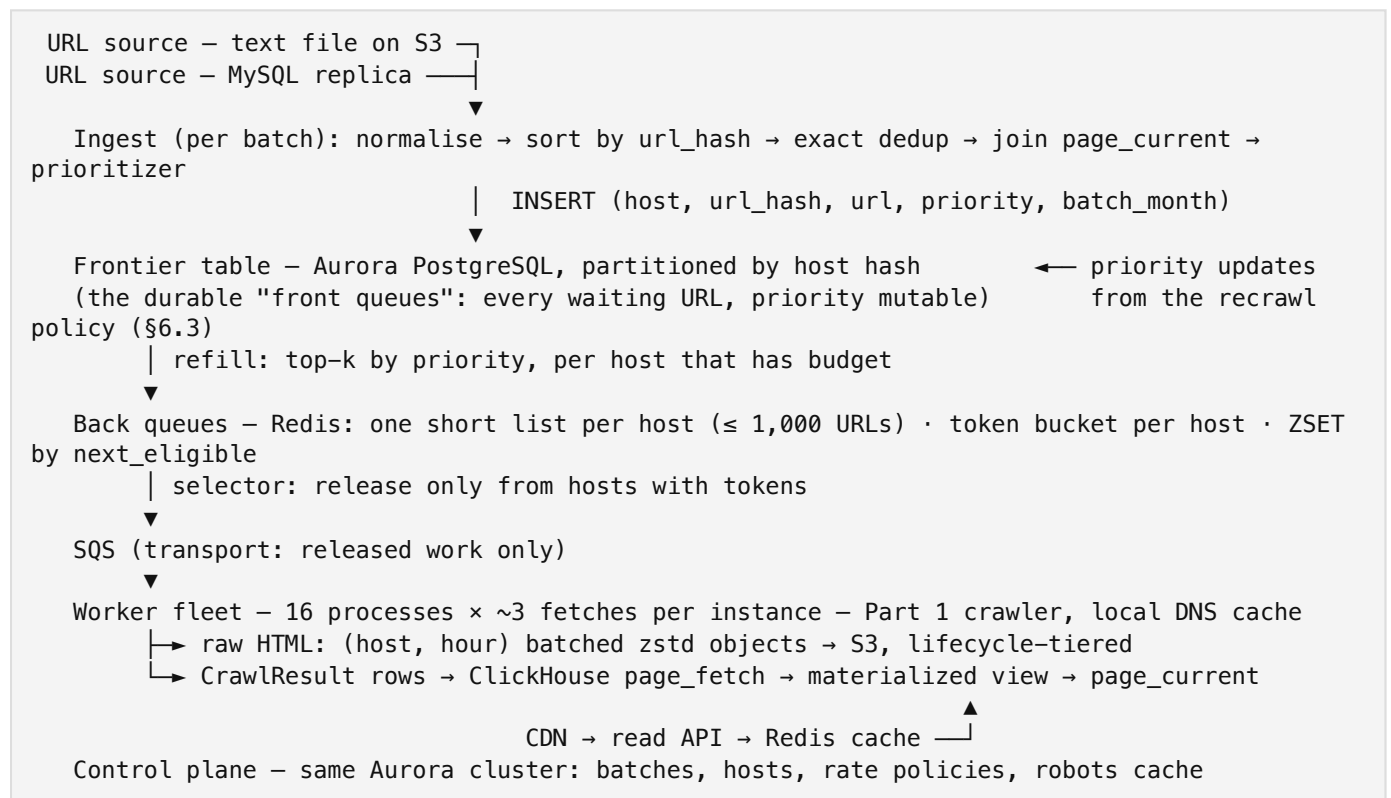
2.2 Few-domain regime

ID	Quantity	Value	Label	Inputs / working
F-00	Hosts	3	assumed	the brief's example

ID	Quantity	Value	Label	Inputs / working
F-01	URLs per host	3.33e9	derived	N-00 / F-00
F-10	Unnegotiated polite rate	10 req/s per host	assumed	no standard exists; common crawler defaults are 1 req/s or the host's Crawl-delay; 10 is the upper end tolerated without an agreement
F-010	At 10 req/s/host: URLs per month · years to finish one host · cores	77.8M · 10.6 y · 22.5	derived	$3 \times 10 \times 2,592,000$; $3.33e9 / (10 \times 31,536,000)$; 30×0.750 s
F-100	At 100 req/s/host (negotiated)	777.6M · 1.1 y · 225.0	derived	same with $r = 100$
F-001	At 1 req/s/host	7.8M · 105.7 y · 2.2	derived	same with $r = 1$

The two regimes differ by two orders of magnitude in fleet and by four in completion time. In the few-domain regime no fleet size changes the outcome: the ceiling is the rate the three hosts accept, which is a negotiation, and the design problem becomes choosing the 0.78% of the corpus (25.9M of 3.33B per host per month, derived from F-010) that is worth the budget. §6 designs that choice.

3. Architecture



3.1 Ingest

The monthly list arrives as a file ("text file") or a table ("MySQL"). Both paths converge after normalisation.

- **File.** Land on S3; workers read byte ranges of the 800 GB object in parallel (N-28).

- **MySQL.** Read from a replica, paged by key (`WHERE id > :last LIMIT 10000`), never by `OFFSET`, which makes MySQL walk every skipped row.
- **Normalise.** Lowercase host, strip fragment, drop tracking parameters (`utm_*`, `ref=`, session ids), sort the rest, reject URLs over 2 KB (a spider-trap guard). The Amazon test URL carries five such parameters and normalises to `/dp/B009GQ034C`. `url_hash` is a 64-bit hash of the normalised URL.
- **Dedup and policy.** §6.1.
- **Prioritise and load.** Each URL gets an initial priority (§6.3) and is bulk-loaded (`COPY`, ~100k rows per batch) into the frontier table: $10e9 \text{ rows} \times 120 \text{ B} = 1.2 \text{ TB}$ (N-29). This is a sort-and-load of 800 GB once per batch (Spark on EMR, or ClickHouse reading the file), not per URL.

Ingest is a throughput component: it must finish before the fleet drains the previous batch, or the fleet idles.

3.2 Frontier

The frontier follows the URL-frontier pattern described in Alex Xu, *System Design Interview* (vol. 1, ch. 9, "Design a web crawler"): a **prioritizer** feeding **front queues** ordered by priority, a **router** into **back queues** — one per host — that enforce politeness, a **selector** that hands workers URLs only from hosts whose delay has elapsed, and hybrid storage with most of the frontier on disk and only queue heads in memory. A plain queue cannot do this: SQS has no consumer-side rate control, and a message for a host with no budget would be consumed and re-queued (paid, repeatedly) or block its consumer.

Pattern component	Here	Property that decides the store
Prioritizer	ingest assigns the initial priority; the recrawl job (§6.3) updates it in place	priority must be mutable after ingest
Front queues	frontier table <code>frontier(host, url_hash, url, priority, next_eligible, released_at, batch_month)</code> in Aurora PostgreSQL, partitioned by host hash, index <code>(host, released_at, priority DESC)</code>	durable; <code>ORDER BY priority DESC LIMIT k</code> is an index walk; <code>UPDATE priority</code> is cheap; 1.2 TB (N-29) fits one cluster
Back queues (one per host)	Redis list per host holding the next $\leq 1,000$ URLs, refilled from the table only when the host has budget	in-memory heads only; worst case $1\text{M hosts} \times 1,000 \times 100 \text{ B} = 100 \text{ GB}$, in practice a small fraction because only budgeted hosts are refilled
Politeness	Redis token bucket per host + ZSET of hosts by <code>next_eligible</code> (DR-10)	atomic sub-millisecond take
Selector	scheduler service (replicated): pops hosts with tokens, <code>LP0P</code> s that many URLs, <code>SendMessageBatch</code> to SQS, sets <code>next_eligible</code>	only budgeted work is ever in transport
Worker	Part 1 crawler, one process per vCPU (§3.3)	—

Scheduler loop. **Refill:** for each host whose back queue is below 200 URLs and whose bucket has tokens, `SELECT ... WHERE host = ? AND released_at IS NULL ORDER BY priority DESC LIMIT 1000`, set `released_at`, `RPUSH`. **Select:** pop hosts with `next_eligible ≤ now` from the ZSET, take up to `tokens`, `LP0P` that many, enqueue in batches of 10, set `next_eligible = now + n / rate`. A host with no budget is neither refilled nor selected; its URLs stay in the table with their priorities, and the batch-progress metric (§11) shows it as *host-limited* rather than *fleet-limited*. A priority change takes effect at the host's next refill: at most 1,000 URLs per host sit in a back queue at any time.

Feedback. Workers report per-host outcomes. A 429, or a rise in a host's 5xx or latency, halves that host's rate; sustained 2xx restores it additively (AIMD), bounded above by the host's policy cap from the control plane (its `Crawl-delay`, a negotiated rate, or F-10). `robots_state = disallowed` from the Part 1 result stops the URL, not the host.

Durability. The frontier table is the durable state. Redis holds only back-queue heads and buckets, both rebuildable: after a Redis loss, rows with `released_at` set but no `page_fetch` row within 1 h are re-released, and the work is idempotent (§9). No cursor checkpoint is needed (DR-15).

Rejected shapes. One SQS queue per host — ~1M queues and paid empty polls. Immutable per-host sorted files on S3 with a cursor — cannot express a priority that changes after ingest, which §6.3 requires hourly. Everything in Redis — 10^9 URLs $\times$ 100 B $\approx$ 1 TB of RAM.

3.3 Process model

Measured on the development machine (CNN fixture, 4 parses, $n=3$): with the parse called directly inside the coroutine, wall time is 1.36–1.47 s and the event loop is frozen for the whole run; with the parse in worker threads (`asyncio.to_thread`), wall time is 0.76–0.79 s and the longest loop stall is 98–158 ms. The parser releases the interpreter lock for part of its work (its C parsing layers), so threads give partial, sub-linear parallelism (1.8 $\times$ on 4 threads here) and, more importantly, keep the loop serving I/O.

Two consequences. The Part 1 service runs parse and topic extraction in a worker thread, so a request in flight during another request's parse still has its socket serviced and `/health` keeps answering; the demo deployment still caps concurrency at 2 because the instance has one vCPU. The fleet runs **one process per vCPU**, each with its own event loop and ~3 in-flight fetches (N-26: 43 in flight per 16-vCPU instance): the thread speedup is sub-linear and unmeasured on the target core, and process-per-core needs no such assumption. Fetching is I/O-bound and needs few sockets; parsing is the CPU constraint and sets the fleet.

Memory follows from the process model: 400 MB per process after a parse $(N-25) \times 16 = 6.4$ GB per instance, so a 16-vCPU / 32 GB compute-optimised instance carries the fleet with headroom; memory-optimised instances would be waste.

Each instance runs a local caching DNS resolver: at 3,858 fetches/s across ~1M hosts the fleet would otherwise issue lookups at a rate the VPC resolver limits per network interface (1,024 packets/s per ENI, AWS documented limit) and pay a resolver round trip per fetch.

3.4 Peak and autoscaling

N-02 gives a 7,716/s peak if arrivals were diurnal. In the many-domain regime the frontier releases work at the rate the fleet drains it, so arrivals are flat and the fleet is sized for the mean (N-07 at 70% utilisation: 258 instances). The autoscaling rule is: oldest released message > 1 h $\rightarrow$ add 20% capacity, up to 1.5 $\times$ mean; below 15 min for 2 h $\rightarrow$ remove 10%. The batch-completion SLO (§10) absorbs the remainder. In the few-domain regime there is no fleet peak: the host budget is the ceiling and 2 instances cover it (F-20).

3.5 Check against the reference pattern

Component by component against the crawler design in Xu (ch. 9), with the deviations and why:

Reference component	This design	Deviation
Seed URLs	the customer's monthly list (the brief's input)	no discovery crawl: URLs are supplied
URL Frontier (prioritizer, front/back queues, selector)	§3.2	disk tier is a relational table rather than files, so priorities are mutable
HTML Downloader (robots.txt, timeouts, retries)	Part 1 <code>service_download</code> , <code>service_robots</code> ; robots cached per host with TTL	robots body capped at 512 KiB per RFC 9309
DNS Resolver (cache)	per-instance caching resolver (§3.3)	—
Content Parser	Part 1 <code>service_parse</code> , <code>service_pagetype</code> , <code>service_topics</code>	classification and topics are outputs, not filters
Content Seen? (hash of page)	raw-body hash <code>content_hash</code> (§6.1)	—
Content Storage	S3 raw objects + ClickHouse body (§4)	two stores for two access patterns
URL Extractor, URL Filter	ingest normalisation; no link extraction	the list is the universe; a 2 KB URL length cap guards traps
URL Seen? (Bloom filter or hash table)	exact sort + join at ingest (§6.1)	Bloom rejected: 1% FPR drops 100M URLs/month silently (N-18)
URL Storage	frontier table + <code>page_current</code>	—
Distributed crawl by geography	single region in v1; per-host egress region is a §14 item	not needed for three US hosts
Server-side rendering	excluded and measured (DR-24)	—
Anti-spam / spider traps	URL length cap; per-host URL cap per batch; <code>robots_state</code> and block reasons recorded	no link graph, so no trap loops
Extensibility	extra map, <code>extractor_version</code> , <code>reason</code> enum	—

4. Storage — three workloads, and where content lives

Workload	Data	Volume (many-domain)	Store
Transactional + frontier	batches, hosts, rate policies, robots cache; the frontier table	~100 GB + 1.2 TB (N-29)	Aurora PostgreSQL (DR-14, DR-15)
Write-once, read-rarely	raw HTML	400 TB/month compressed (N-11)	S3, batched objects, lifecycle-tiered (DR-12, DR-23)
Analytical scan + point lookup	metadata and extracted body	25 TB/month compressed (N-10b)	ClickHouse, two tables (DR-13, DR-21)

Content is stored twice, for two reasons. The extracted body is a column in ClickHouse, because it is what customers query and what topic extraction consumed. The raw HTML is in S3, because extraction will be re-run: a parser fix or a new field should not cost a re-fetch, and `extractor_version` on every row says which rows a re-run must touch. Raw HTML is written as one object per (host, hour) holding up to 1,000 pages as concatenated zstd frames; a page is addressed by `s3_key`, `s3_offset`, `s3_length` and read with one ranged GET. That keeps the PUT rate at 3.9 PUT/s instead of 3,858 PUT/s, under S3's per-prefix limit, and the PUT bill at \$50/month instead of \$50,000 (N-14, N-14b).

Lifecycle: Standard for 30 days (re-extraction window), Infrequent Access to 90 days, Glacier Deep Archive after. Storage is a stock, not a flow; §12 shows the bill at 1, 12 and 24 months.

5. Unified schema

One schema for every page type. Type-specific fields go in extra, not in per-type tables, so a product page and an article are rows in the same table with the same core columns.

```
-- every fetch, including blocked and error outcomes; append-only
CREATE TABLE page_fetch (
  batch_month      Date,                -- the month the customer asked for
  fetched_at       DateTime64(3),
  domain           LowCardinality(String),
  url_hash         UInt64,              -- of the normalised URL
  url              String,
  final_url        String,
  status           LowCardinality(String), -- success | blocked | error
  reason           LowCardinality(String), -- Part 1 Reason enum
  robots_state     LowCardinality(String), -- allowed | disallowed | missing | unavailable
| unreachable | skipped
  http_status      Nullable(UInt16),
  content_type     LowCardinality(String),
  content_hash     UInt64,              -- of the raw response body; change detection
  title            String,
  description       String,
  canonical_url    String,
  author           String,
  published_date   Nullable(DateTime),
  language         LowCardinality(String),
  h1_headings      Array(String),
  og_tags          Map(String, String),
  twitter_tags     Map(String, String),
  body             String CODEC(ZSTD(3)), -- extracted content
  word_count       UInt32,
  page_type        LowCardinality(String), -- product | article | listing | profile |
other | unknown
  topics           Nested(topic String, rank_score Float32),
  s3_key           String,              -- batched raw object
  s3_offset        UInt32,
  s3_length        UInt32,
  extractor_version LowCardinality(String),
  extra            Map(String, String)  -- price, sku, rating ...
)
ENGINE = MergeTree
PARTITION BY toYYYYMMDD(fetched_at)
ORDER BY (domain, url_hash, fetched_at);

-- one row per URL: the latest fetch; fed by a materialized view from page_fetch
CREATE TABLE page_current AS page_fetch
ENGINE = ReplacingMergeTree(fetched_at)
PARTITION BY cityHash64(url_hash) % 64
ORDER BY (domain, url_hash);
```

Why two tables. `ReplacingMergeTree` collapses duplicates only inside one partition, so a single table partitioned by time keeps every recrawl as a separate row and "latest metadata for this URL" needs `FINAL` across partitions. `page_fetch` is the history and the audit trail (which hosts blocked when, which extractor produced what), partitioned by fetch day so a day's parts merge among themselves and old days expire by

TTL. `page_current` is partitioned by URL hash so the same URL always lands in the same partition and Replacing collapses it; its size is bounded by distinct URLs, not by fetch count. Both tables order by `(domain, url_hash)`, which serves the month/domain scan implied by the brief's input example (one domain range inside `batch_month`) and the point lookup (one URL: one granule).

`batch_month` is distinct from `fetched_at`: the customer asks for July's list; a URL from that list may be fetched in August after a backlog, and a URL may appear in July's and August's lists.

Partitioning alternatives and why not: a single monthly partition holds 10e9 rows / 25 TB (N-10b) and merges at that granularity; partitioning by domain creates ~1M partitions and stalls inserts with too many parts. Decision record DR-21.

6. Dedup, politeness, recrawl and prioritisation

6.1 Two dedup questions, answered separately

Question	When	Structure	Exactness
Seen in this batch ?	ingest	external sort by <code>url_hash</code> ; drop adjacent duplicates	exact
Crawled before , and does policy want it again?	ingest	batch join of the sorted list against <code>page_current (url_hash, fetched_at, content_hash)</code> ; DR-25 decides	exact
Content unchanged since last fetch?	after fetch, before parse	<code>content_hash</code> = hash of the raw response body; equal to the stored hash → skip parse and write, update <code>fetched_at</code> only	exact

A Bloom filter is not used: at 1% false-positive rate it would silently drop 100M URLs per month as false duplicates (N-18) and, having no delete, would mark a URL crawled in July as "seen" forever. The batch join costs one large query per batch instead of a per-fetch lookup (DR-20).

At an assumed 60% unchanged rate on recrawl, skipping parse and write removes 60% of compute — \$25,000/month at N-09 — and 60% of writes (N-17). The hash is of raw bytes because the parse is what the skip avoids, and that makes the 60% an upper bound: a host that serves per-fetch variants never hashes equal. Amazon did exactly that on the three recorded fetches (319, 1,154 and 1,487 words of extracted body from the same URL), so for the brief's own hosts the unchanged rate may be far below 60%; the PoC's second crawl cycle measures it per host, and the saving is quoted as a range from that measurement, not as this planning value.

6.2 Politeness

- `robots.txt` fetched once per host, cached with a TTL (24 h; 5 min for `unavailable`), applied per URL; `robots_state` is stored on every row.
- Per-host token bucket in the frontier (§3.2); the queue never holds a URL whose host has no budget.
- AIMD on 429 and rising 5xx; a host's policy cap is its `Crawl-delay`, a negotiated rate, or F-10.
- One identifying `User-Agent` with a contact URL. Blocks are recorded, not evaded (§8).

6.3 Recrawl and prioritisation under budget

In the few-domain regime the monthly budget per host is 25.9M fetches (F-010) against 3.33B URLs, so 99.2% of the list is not fetched in any given month. The order in which the frontier releases a host's URLs is therefore the product:

```

priority = customer_weight(url)          -- supplied with the list, or 1.0
        × staleness(url)                -- (now - fetched_at) / interval(url), capped at 4;
never-fetched = 2
interval(url): starts at a page_type prior (article 7 d, product 30 d, listing 14 d, other 30
d);
                doubles when content_hash is unchanged, halves when it changed; bounded 1 d –
90 d

```

Never-fetched URLs and recrawls share the budget at a configurable split (default 50/50) so a large first list does not starve freshness and a mature list does not starve discovery. The recrawl job writes `priority` into the frontier table hourly; the back queues pick the change up at the next refill (§3.2). Fixed cadences ("news hourly") are rejected: hourly is 720 fetches per URL per month and at any budget it starves the rest of the host (DR-25). Completion in this regime is reported as *coverage of the list at the negotiated rate*, with an ETA per host, not promised as a percentage of the list.

7. Serving millions of requests

Read traffic is assumed at 10M requests/day: 116/s mean, 231/s at a 2× peak (N-20). Two access patterns:

Pattern	Example	Path
Point lookup	metadata for one URL	CDN → API → Redis cache → <code>page_current</code> by <code>(domain, url_hash)</code>
Analytical	all amazon.com pages for July, aggregates	API → <code>page_current</code> or <code>page_fetch</code> with <code>batch_month</code> filter

A full-key lookup on `page_current`'s sorting key reads one granule; the column store is not scanning a partition to answer it. With a 90% cache hit (assumed) the store sees 11.6/s of misses (N-20). A key-value replica of every record is not part of the design: writing 10e9 records of 10 KB into DynamoDB is 100 billion write units, \$125,000/month (N-21), plus \$25,000/month of storage accumulating (N-21b), to serve about 12 requests per second. If the measured p99 of point lookups on ClickHouse under 250 concurrent readers while inserts run exceeds 100 ms, a KV tier holding only a ≤1 KB summary per URL returns at one write unit per record, \$12,500/month (DR-16).

Cache TTL equals the URL's recrawl interval; a write to `page_current` invalidates the key. The CDN caches by normalised URL parameter with a short TTL and terminates TLS (DR-17, DR-18).

8. Blocking and JavaScript rendering as coverage constraints

Measured on the brief's three test URLs (Part 1 crawler, 2026-09-13, one fetch each from a residential IP; repeated 2026-09-14 from the Cloud Run deployment with the same status outcomes):

URL	Outcome	Detail
Amazon product	success, product	HTTP 200; 1.6 MB; the 2026-09-13 residential fetch carried no <code>Content-Type</code> header (body sniffed), the recorded Cloud Run fetch carried <code>text/html; charset=UTF-8</code> ; extracted word count differed on every fetch (319 / 1,154 / 1,487)
CNN article	success, article	HTTP 200, 5.6 MB

URL	Outcome	Detail
REI blog	blocked, forbidden	HTTP 403 from the Akamai edge for the page and for /robots.txt; 200 for a Safari User-Agent from the same machine

Two consequences for the design:

Pages that are not the content are detected generically. A 200 with under 100 extracted words and no declared metadata is `error / no_content`, whatever its wording or language; phrase lists only upgrade such a page to `blocked` when it names itself a denial. Disguised blocks land there too: Myntra served a "Site Maintenance" page with HTTP 200 to the crawler's User-Agent and the product page to a Safari User-Agent (measured 2026-09-14, one fetch each). Two host-level signals a single fetch cannot have separate them: the same `content_hash` across many distinct URLs of one host is a template page, and a browser-User-Agent probe that succeeds where the crawler got `no_content` is a block (Part 3 B13). Hosts where the probe disagrees are counted as blocking. A 52-URL smoke run on 2026-09-14 (README, "Behaviour on other sites") shows the shapes this takes in practice: a 200 maintenance page (Myntra), an empty 202 challenge (Ars Technica), a TLS reset (Best Buy), a CAPTCHA with 200 (Walmart).

Block rate is a per-host measurement, never an assumption. The Part 1 result separates `blocked` (the host refused: `robots_disallowed`, `captcha`, `forbidden` incl. 401/403/451, `rate_limited`) from `error` (something failed: DNS, connect, timeout, 5xx, `robots_unavailable`, `not_found`, `redirect_not_followed`, `not_html`, `too_large`, `parse_failed`, `private_address` for a target the crawler refuses to fetch, and `no_content` for a 200 with almost no visible text and no declared metadata). Block rate is a coverage number; error rate is a reliability number; they are reported separately everywhere (§10, §11). Passing a User-Agent block means misrepresenting the crawler, so it is a policy decision the customer makes, and the default is to record the block and report the coverage ceiling it sets.

JavaScript-only pages are excluded in v1 and measured. A sampled detector flags pages where the extracted body is empty, the HTML exceeds 100 KB and `<script>` count is high; the share per host is reported with each batch. Headless rendering is costed as a separate workstream at an assumed 10–20× the CPU of a static parse; if the measured share on target hosts exceeds 10%, a rendering pool is budgeted from that measurement (DR-24). SLA language (§10) excludes JS-only pages and reports their count.

9. Failure modes and idempotency

One URL is an idempotent unit: the same fetch produces the same `page_fetch` row keyed by (`url_hash`, `fetched_at`) and the same `page_current` row keyed by `url_hash`, and `ReplacingMergeTree` collapses a duplicate write. At-least-once delivery is therefore safe, and the recovery for most failures is "release the URL again".

Failure	Detected by	Response
Worker dies mid-fetch	SQS visibility timeout	message reappears; the frontier's budget already paid for it, so no re-take
Spot instance reclaimed	2-minute notice	stop taking messages, flush the raw-object buffer, exit
Raw-object buffer lost before flush	process crash	rows for those pages are written only after the object PUT succeeds, so the URLs are released again next cycle; loss is ≤ 60 s of one worker's pages
Redis loss	health check	buckets rebuilt from policy; back queues refilled from the frontier table; rows released > 1 h ago without a <code>page_fetch</code> row are re-released, idempotent

Failure	Detected by	Response
Frontier table unavailable	scheduler query errors	no refill; back queues drain in $\leq 1,000$ URLs per host, then the fleet idles — visible as a release-rate drop, nothing lost
A URL crashes the parser	receive count > 3	dead-letter queue; <code>status = error</code> , <code>reason = parse_failed</code> row written; fleet unaffected
ClickHouse insert lag	insert queue depth	workers buffer up to 5 min then back-pressure the scheduler (release rate lowered); nothing dropped
Host starts blocking	per-host block rate	AIMD lowers rate; alert; coverage report updated
AZ loss	health checks	multi-AZ instances; stateless workers; Redis and Postgres replicated
Bad deploy	error-budget burn	roll back; freeze deploys

Back-pressure flows toward the frontier, which is the only component that can slow the system without losing work: a growing per-host file cursor gap is visible and recoverable; a dropped message is neither.

10. SLOs and SLAs

SLOs are internal and measured continuously; SLAs are external and looser. They differ by regime because the few-domain regime's completion is bounded by the host budget, not by the system.

SLO	Many-domain	Few-domain	Measured by
Release-to-fetch latency	99% of released URLs fetched within 1 h	same	<code>fetched_at - released_at</code>
Batch completion	99% of fetchable URLs in a monthly list fetched within that month + 24 h	$\geq 95\%$ of each host's negotiated daily budget consumed	control-plane batch table
Parse latency	p95 < 1.5 s on production cores (N-05b mean 750 ms; p95 assumed 2x mean)	same	per-crawl trace
Permanent error rate (<code>status = error</code> , excluding blocked)	< 0.5% of fetched	same	<code>page_fetch</code>
Row visibility	a fetch is queryable in <code>page_current</code> within 10 min	same	insert lag
Read API	99.9% availability; p95 < 200 ms cached, < 1 s on miss	same	API metrics

SLA (external)	Commitment
Read API availability	99.5% monthly
Batch completeness	95% of fetchable URLs; fetchable excludes <code>blocked</code> , <code>robots_disallowed</code> and JS-only pages, each reported by count per host with the batch
Few-domain coverage	delivered as coverage at the negotiated rate with a per-host ETA; no percentage-of-list commitment

SLA (external)	Commitment
Freshness	a fetched page is queryable within 1 h

"Fetchable" is load-bearing: committing to a share of *all* URLs would be committing to defeat bot detection.

11. Monitoring — metric, threshold, action, tool

Per-host series are computed in ClickHouse from `page_fetch`, not emitted as metric labels: ~1M hosts as a label would make a time-series database unusable and expensive (DR-19). Fleet metrics go to Prometheus; AWS-native signals to CloudWatch.

Metric	Threshold	Action	Tool
Batch progress: fetched / fetchable, and per-host ETA = remaining / current rate	ETA past batch end + 24 h	if fleet-limited: scale; if host-limited: report to customer, review rate policy	control plane + Grafana
Oldest released message age	> 1 h	scale workers +20% (§3.4)	CloudWatch (SQS)
Per-host block rate (<code>blocked</code> / fetched)	> 2x the host's 7-day baseline	AIMD already applied; open a coverage ticket; customer report	ClickHouse query → Grafana
Per-host error rate (<code>error</code> / fetched)	> 5% for 15 min	check DNS/TLS/host health; hold host if 5xx	ClickHouse → Grafana
Per-host <code>no_content</code> rate, and distinct URLs sharing one <code>content_hash</code>	> 1%; > 100 URLs on one hash	probe the host with a browser User-Agent; if the probe succeeds, reclassify the host as blocking and open a coverage ticket; a shared hash is a template page	ClickHouse → Grafana
<code>robots_unavailable</code> count per host	> 0 for 1 h	host is unwell; hold host	ClickHouse
Parse p95	> 1.5 s	page-size growth or parser regression; profile the top hosts by size	OpenTelemetry traces (sampled)
Parse failure rate	> 1%	site redesign or parser bug; check <code>extractor_version</code>	ClickHouse
Topic extractor failure rate (logged warnings) vs empty-topic rate	failures > 0.1%; empty > 5%	extractor crash vs thin pages — different fixes	logs → Athena; ClickHouse
Dedup hit rate (unchanged / recrawled)	falls > 10 points week over week	normalisation or hashing regression; cost rises directly	ClickHouse
ClickHouse parts per partition; insert lag	> 300 parts; lag > 5 min	increase insert batch size; back-pressure scheduler	ClickHouse system tables → Grafana
Spot interruption rate	> 5%/h	shift 20% to on-demand	CloudWatch (ASG)
Cost per million URLs	> 20% over model	check dedup rate, page size, instance mix	Cost Explorer, tagged by component
SLO error-budget burn	> 2x expected	freeze deploys	Grafana

Every worker emits one terminal structured JSON event per crawl — `crawl.ok`, `crawl.blocked` or `crawl.failed` — carrying `url`, `reason`, `robots_state` and `http_status` (`service_crawl._finish`); that event and the `CrawlResult` it accompanies are the source for both the ClickHouse rows and the logs.

12. Cost model

Prices are assumed us-east-1 list prices as recalled at writing (Graviton spot \$0.02/vCPU-h; S3 \$0.023 / \$0.0125 / \$0.00099 per GB-month for Standard / IA / Deep Archive; PUT \$0.005 per 1,000; NAT \$0.045/GB; public IPv4 \$0.005/h; SQS \$0.40/M; gp3 \$0.08/GB-month) and are the inputs to `scripts/ledger.py`.

12.1 Many-domain, per million URLs

Line	Working	Per million
Compute	208 core-hours (N-08) × \$0.02	\$4.17
S3 PUT, batched	1,000 PUTs × \$0.005/1,000	\$0.005
Egress via public IPv4 (DR-22)	\$651/month / 10,000	\$0.07
Raw storage, first month	40 GB × \$0.023	\$0.92
Metadata storage, first month	2.5 GB × \$0.08	\$0.20
SQS	300k requests × \$0.40/M	\$0.12
Variable total (public IPv4)		\$5.48
Variable total if egress via NAT Gateway	+ 40 GB × \$0.045 = \$1.80 – \$0.07	\$7.21
Fixed infrastructure, spread over 10e9/month	\$9,293 (N-24e) + \$120 frontier storage (N-29) / 10,000	\$0.94
All lines, month 1	N-24f	\$6.42

N-24 and N-24f in the ledger. Fixed infrastructure (assumed list prices): ClickHouse 6 × i4i.4xlarge \$5,931; Aurora PostgreSQL 2 × r6g.2xlarge \$1,495; ElastiCache 4 × cache.r7g.xlarge \$1,267; read API and CloudFront \$300; observability \$300 — \$9,293/month (N-24e). At 10e9/month the first-month bill is **\$64,181** (N-24f).

12.2 Storage is a stock

Monthly bill	Month 1	Month 12	Month 24
Raw HTML in S3 under the lifecycle (N-12)	\$9,200	\$22,764	\$27,516
Metadata in ClickHouse, gp3 for 3 months then S3-backed (N-13)	\$2,000	\$11,175	\$18,075
Compute + PUT + IPv4 + SQS (variable, N-24g)	\$43,568	\$43,568	\$43,568
Fixed infrastructure + frontier storage (N-24e, N-29)	\$9,413	\$9,413	\$9,413
Flat lines subtotal (N-24c)	\$52,981	\$52,981	\$52,981
Total per month	\$64,181	\$86,920	\$98,572

Cumulative raw HTML: 4.8 PB at 12 months, 9.6 PB at 24. Deleting raw HTML older than the re-extraction window would cap the raw line at the month-3 level; the design keeps it because a re-extraction over history is cheaper than a re-fetch and is the only way to apply a parser fix to old rows.

12.3 The three largest levers

Lever	Effect	Label
Dedup: skip parse and write for unchanged raw content	up to –\$25,000/month compute and –\$30 of PUTs at 60% unchanged (N-17); storage growth falls in proportion	upper bound; per-host rate measured in PoC cycle 2 (§6.1)
Egress path: public IPv4 instead of NAT	\$651 instead of \$18,000/month (N-15b vs N-15); on uncompressed bytes NAT would be \$90,000 (N-15a)	derived
Raw-object batching	\$50 instead of \$50,000/month in PUTs (N-14b vs N-14)	derived

Rejected on cost, for the record: Lambda for the worker, \$64,500/month (N-27) against \$41,667 spot; a DynamoDB replica for serving, \$125,000/month (N-21).

12.4 Few-domain

ID	Line	Value	Label
F-20	Fixed floor: RDS \$173; Redis × 2 \$317; ClickHouse × 2 \$988; observability \$100; 2 worker instances \$461	\$2,038/month	assumed list prices
F-21	Variable at 10 req/s/host: 3.1 TB raw + metadata	\$87.09/month	derived
F-22	Per million URLs at 77.8M/month	\$27.33	derived

The fixed floor dominates; per-URL cost falls as the negotiated rate rises. The number the customer needs in this regime is not cost per million but **coverage per month at the agreed rate** (F-010, F-100), and the cost of raising it is a negotiation, not a fleet.

13. Decision records

Format: what the choice serves, what was chosen, what was rejected and the property that disqualified it, the decisive property, cost shape, the condition that reverses it, what to validate first, and the honest level of experience behind it.

DR-09 Queue (transport)

Serves: deliver released URLs to workers at N-01 (3,858/s many-domain; ≤ 30 /s few-domain)

Chosen: SQS standard, batched 10

Rejected:

- Kafka / MSK – ordering and replay are capabilities this pipeline does not consume; per-host politeness would need partition count to track host count
- RabbitMQ – self-operated broker; per-host queues for politeness means ~ 1 queue per host and paid empty polls on each
- Redis Streams – durability tied to Redis persistence settings; the frontier already relies on Redis for state and should not also be the durable transport

Decisive property: the frontier does rate control, so the queue needs only at-least-once delivery and a visibility timeout – SQS's exact contract

Cost shape: N-16: \$1,200/month batched, \$12,000 unbatched

Reverses if: several consumers need the same fetch event (then a log is right)

Validate first: batch-receive latency at 3,858 msg/s from 181 instances

Experience: architectural choice

DR-10 Frontier state store

Serves: \$3.2 – per-host token bucket, cursor, next-eligible time; ~ 1 M hosts

Chosen: Redis (ElastiCache); Lua-scripted take-token; ZSET of hosts by next_eligible

Rejected:

- DynamoDB – a conditional write per token = 1 WCU per fetch $\rightarrow 10e9 \times \$1.25/M = \$12,500/\text{month}$, and ~ 10 ms per token against sub-millisecond
- PostgreSQL – a token take is a read-modify-write per fetch; 3,858 row updates/s with WAL fsync on one primary is at the edge, and rows are hot in the few-domain regime

Decisive property: atomic read-modify-write in one round trip, sub-millisecond

Cost shape: fixed per node; state is ~ 1 M hosts $\times \sim 200$ B ≈ 200 MB

Reverses if: host count $> \sim 50$ M (shard by host hash), or cursor loss is unacceptable even for 60 s (checkpoint more often – DR-15)

Validate first: token throughput per node; behaviour on failover

Experience: architectural choice

DR-11 Worker compute

Serves: N-06 cores; idempotent work units; cost

Chosen: EC2 Graviton (c7g) Spot in an auto-scaling group, ECS-managed containers, one process per vCPU

Rejected:

- Lambda – N-27: \$64,500/month on per-invocation pricing against \$41,667 spot; no connection reuse across invocations
- Fargate Spot – no control over process-per-core placement; per-vCPU price above EC2 spot like-for-like (assumed $\sim 1.3\times$; verify)
- EKS – cluster fee and operational surface for scheduling needs an ASG already meets

Decisive property: cheapest core-hour; interruption is safe because a URL is idempotent

Cost shape: N-08: 208 core-hours per million URLs

Reverses if: spot interruption rate makes the 1 h release-to-fetch SLO miss (then an on-demand baseline of $\sim 30\%$)

Validate first: parse CPU on c7g – the $1.5\times$ in N-05b

Experience: architectural choice

DR-12 Raw store

Serves: "Design storage of the metadata and content" – raw HTML, write-once, read-rarely
 Chosen: S3, lifecycle Standard (30 d) → Infrequent Access (90 d) → Glacier Deep Archive
 Rejected:
 – EFS – \$0.30/GB-month (assumed list) ≈ 13× S3 Standard; POSIX not needed
 – HDFS on EC2 – 3× replication; storage coupled to compute uptime
 Decisive property: per-GB price with tiering; PUT/GET matches write-once
 Cost shape: N-12: \$9,200 → \$22,764 → \$27,516 per month at 1 / 12 / 24 months
 Reverses if: re-extraction is frequent (keep more in Standard)
 Validate first: compression ratio across target hosts (5:1 measured on n=2)
 Experience: operated at small scale (S3 API)

DR-13 Metadata and content store

Serves: the month/domain scan implied by the brief's input example; body text; point lookup
 Chosen: ClickHouse (self-managed on EC2, or ClickHouse Cloud), two tables (DR-21)
 Rejected:
 – PostgreSQL – a row store reads every column of each matching row; the body column dominates
 GiB bytes. On the same 10M rows of a public reviews dataset, Postgres used 3.84 GiB against ClickHouse's 2.12 GiB (measured, one node, single run). Query-time multiples are not cited: ClickBench (published by ClickHouse; vendor-run)
 gives the direction, and a fair Postgres comparison on target hardware is in §14
 – BigQuery – priced per TB scanned; the crawl runs in AWS, so 100 TB/month of writes
 crosses clouds
 – Redshift – per-node pricing with storage coupled to compute unless RA3; ingest via COPY batches only
 – Elasticsearch – inverted-index storage 2–3× columnar for the same rows; not an aggregate scanner
 – DynamoDB – no analytical scans or aggregates
 Decisive property: columnar compression plus a sorting key serve both the month/domain scan and the single-URL lookup from one store
 Cost shape: N-13 storage stock; compute fixed per node
 Reverses if: 3,858 rows/s of inserts produce unmanageable part counts
 Validate first: sustained insert batching (tens of thousands of rows per insert) and merge lag
 Experience: operated at small scale (single node, 10M rows); not in production

DR-14 Control-plane store

Serves: batches, hosts, rate policies, robots cache (~1M host rows) and the frontier table (10e9 rows, 1.2 TB – N-29)

Chosen: Aurora PostgreSQL (multi-AZ), frontier partitioned by host hash

Rejected:

- DynamoDB – scheduling reads are multi-row predicates ("hosts in batch X with cursor < end")
needing a secondary index per predicate; no joins
- MySQL – equivalent for this workload; Postgres chosen for partial indexes on "hosts with pending work" and transactional DDL. The brief's source MySQL is an input read from a replica, not this store.

Decisive property: transactions and joins across batch, host and policy rows

Cost shape: fixed instances (N-24e) + \$0.10/GB-month storage (N-29: \$120/month)

Reverses if: the frontier outgrows one cluster's storage limit or refill queries exceed ~1M rows/s (shard by host hash across clusters)

Validate first: scheduler query latency at 1M hosts

Experience: operated at small scale

DR-15 Frontier storage and durability

Serves: §3.2 – where waiting URLs live, and what survives a Redis loss

Chosen: hybrid, after Xu's pattern: the frontier table in Aurora PostgreSQL is the durable store of every waiting URL with a mutable priority; Redis holds only per-host back-queue heads ($\leq 1,000$ URLs) and token buckets, both rebuildable

Rejected:

- immutable per-host sorted files on S3 + a cursor – a priority that changes after ingest (\$6.3, hourly) cannot be expressed; re-sorting an 800 GB object per change is not a plan
- the whole frontier in Redis – $10e9 \times \sim 100 \text{ B} \approx 1 \text{ TB}$ of RAM (assumed row size)
- Redis AOF as the durability story – a failed-over replica may lag; the table needs no checkpoint

Decisive property: mutable priority on disk, sub-millisecond politeness in memory, and nothing to

Cost shape: checkpoint: released rows without a page_fetch row are simply re-released
N-29: 1.2 TB at \$0.10/GB-month = \$120/month plus the Aurora instances in N-24e

Reverses if: refill queries (top-k per host) cannot sustain the release rate – then per-host heads are pre-materialised by a batch job into the back queues

Validate first: refill query latency at 1M hosts with 1.2 TB; re-release correctness after a forced Redis failover

Experience: architectural choice

DR-16 Serving and point lookups

Serves: "allow millions of requests on the content" – N-20: 116/s mean, 231/s peak
 Chosen: CDN → API → Redis cache → ClickHouse page_current by (domain, url_hash); no
 KV replica
 Rejected:
 – DynamoDB replica of every record – N-21: \$125,000/month in writes plus \$25,000/month storage
 accumulating, to serve ~12 misses/s
 – serving from page_fetch – latest-row queries need FINAL/argMax across partitions
 Decisive property: a full sorting-key lookup reads one granule; misses after cache are ~12/s
 Cost shape: fixed (API tier, Redis); zero per-record write cost
 Reverses if: measured p99 of point lookups under 250 concurrent readers while inserts run
 > 100 ms – then a KV tier of ≤ 1 KB summaries at 1 WRU/record
 (\$12,500/month)
 Validate first: that p99
 Experience: architectural choice

DR-17 Cache

Serves: repeat reads; TTL = recrawl interval
 Chosen: Redis (ElastiCache), separate cluster from the frontier
 Rejected:
 – Memcached – a second engine to operate for no capability the design uses
 – CDN only – cannot invalidate per URL on recrawl without a purge call per row
 Decisive property: per-key TTL plus explicit invalidation on the page_current write
 Cost shape: fixed per node
 Reverses if: hit rate measured < 60% – DR-16 still holds; the cache stops being the reason
 Validate first: hit rate on real read traffic
 Experience: operated at small scale

DR-18 CDN

Serves: edge caching and TLS for the read API
 Chosen: CloudFront, cache key = normalised URL parameter, short TTL
 Rejected:
 – none – every read hits the API tier; no edge TLS or WAF
 – Cloudflare – outside the three clouds the brief names as allowed services
 Decisive property: same account, per-request pricing, WAF attach point
 Cost shape: per 10k requests plus egress
 Reverses if: reads become authenticated and uncacheable
 Validate first: edge hit ratio against Redis hit ratio
 Experience: architectural choice

DR-19 Observability

Serves: "key monitoring metrics and tools ... track the system's progress"

Chosen: Amazon Managed Prometheus + Grafana for fleet metrics; OpenTelemetry traces on sampled crawls; CloudWatch alarms for SQS/ASG; per-host metrics as ClickHouse queries over page_fetch; structured JSON logs to S3, queried with Athena

Rejected:

- Datadog - per-host pricing × 181–344 hosts; outside the named clouds
- CloudWatch custom metrics per host - per-metric pricing × ~1M hosts

Decisive property: per-host cardinality belongs in a column store, not in time-series labels

Cost shape: fixed plus per-sample; logs per GB

Reverses if: few-domain regime only - CloudWatch alone suffices

Validate first: Prometheus series count with instance-level labels only

Experience: operated at small scale (Prometheus, Grafana)

DR-20 Dedup structure

Serves: §6.1 - "seen in this batch" and "crawled before" as separate questions

Chosen: exact: sort the batch by url_hash, drop adjacent duplicates; batch join against page_current (url_hash, fetched_at, content_hash); DR-25 decides recrawl

Rejected:

- Bloom filter - 1% FPR silently drops 100M URLs/month (N-18); no delete, so a July URL is "seen" forever
- Redis SET of url_hash - $10e9 \times (16 \text{ B} + \text{overhead}) \approx 500 \text{ GB RAM}$ (assumed)

Decisive property: exactness; batch-time cost instead of per-fetch cost

Cost shape: one sort and one join per batch over 800 GB (N-28)

Reverses if: URLs arrive as a stream rather than a monthly list (then a per-fetch check)

Validate first: join time of $10e9$ rows against page_current

Experience: architectural choice

DR-21 Schema, versioning and partitioning

Serves: "Design for unified data schema"; the month/domain scan; the point lookup; recrawl history

Chosen: page_fetch: MergeTree, PARTITION BY toYYYYMMDD(fetched_at), ORDER BY (domain, url_hash, fetched_at)
page_current: ReplacingMergeTree(fetched_at), PARTITION BY cityHash64(url_hash) % 64, ORDER BY (domain, url_hash), fed by a materialized view

Rejected:

- one table, PARTITION BY month - $10e9$ rows / 25 TB in one partition; Replacing collapses only inside a partition, so recrawls across months never collapse
- PARTITION BY (month, domain) - ~1M partitions; too many parts; insert stalls

Decisive property: history partitioned by ingest time (TTL by day); current state partitioned by URL hash so the same URL always lands where Replacing can collapse it

Cost shape: page_fetch grows with fetches (N-10b); page_current is bounded by distinct URLs

Reverses if: fetch history is never queried (keep page_fetch for 90 days only)

Validate first: parts per partition at 3,858 rows/s with daily partitions

Experience: operated at small scale

DR-22 Egress path

Serves: bytes fetched from the internet by the worker fleet
 Chosen: workers in a public subnet with public IPv4 and egress-only security groups
 Rejected:
 - NAT Gateway – N-15: \$0.045/GB on 400 TB wire bytes = \$18,000/month (+ hourly gateway fees)
 - NAT instances – self-operated; single-AZ failure domains
 Decisive property: public IPv4 at \$0.005/h × 181 instances ≈ \$650/month (N-15b)
 Cost shape: per instance-hour, not per byte
 Reverses if: a partner requires a short allow-list of egress IPs (NAT with Elastic IPs)
 Validate first: block rate by egress IP
 Experience: architectural choice

DR-23 Raw-object batching

Serves: \$4 – PUT rate and PUT cost
 Chosen: per-worker buffer flushes one object per (host, hour) at 1,000 pages or 60 s;
 zstd frame per page; pointer = s3_key + s3_offset + s3_length
 Rejected:
 - one object per page – N-14: \$50,000/month; 3,858 PUT/s on hot prefixes
 - prefix sharding only – fixes the rate, not the \$50,000
 Decisive property: 1,000× fewer PUTs; a single page is one ranged GET
 Cost shape: N-14b: ~\$50/month; ~40 MB objects
 Reverses if: single pages are rewritten often (pages are immutable here)
 Validate first: ranged-GET latency for re-extraction jobs
 Experience: architectural choice

DR-24 JavaScript-rendered pages

Serves: \$8 – coverage constraint and SLA wording
 Chosen: excluded in v1; sampled detector (empty body AND html > 100 KB AND script-heavy)
 reports the share per host; rendering costed as a separate workstream at an assumed 10–20× CPU per page
 Rejected:
 - render everything – fleet × 10–20 on the largest cost line
 - render nothing and not measure – an unknown coverage gap hidden behind the SLA
 Decisive property: the share is measurable in the PoC before any spend
 Cost shape: share × 10–20 × N-05
 Reverses if: measured share on target hosts > 10%
 Validate first: that share on the PoC sample
 Experience: architectural choice

DR-25 Recrawl and prioritisation

Serves: §6.3 – the budget is a fraction of the corpus in the few-domain regime
 Chosen: per-URL adaptive interval (double on unchanged raw-body content_hash, halve
 on change;
 staleness; 1 d – 90 d) inside a per-host budget; release order by customer_weight ×
 never-fetched and recrawl share the budget 50/50 by default
 Rejected:
 – fixed cadence per page type – "news hourly" is 720 fetches per URL per month; at any
 budget
 it starves the rest of the host
 – uniform monthly – no freshness for changing pages; wasted fetches on static ones
 Decisive property: spend follows observed change and stated value inside the host ceiling
 Cost shape: fetches = Σ host budgets, not corpus size
 Reverses if: the customer supplies a per-URL priority (use it directly)
 Validate first: change rate per page_type over two PoC cycles
 Experience: architectural choice

14. What to validate first

Ranked by how much the design changes if the assumption is wrong.

1. **Per-host accepted rate on the target hosts** (F-10). It decides the regime, the fleet, and what can be promised. Measure by crawling each host at 1, 5 and 10 req/s for an hour and recording 429/403 rates.
2. **Page-size distribution** (N-03). Drives storage and bandwidth; at 1 MB the raw store is 2 PB/month compressed and the fleet is network-heavy.
3. **Parse CPU on the target instance type** (N-05b). Scales the largest cost line directly.
4. **Unchanged rate on recrawl** (N-17's 60%). The largest saving; needs two crawl cycles.
5. **Block rate per host, and JS-only share per host** (§8). Sets the coverage ceiling and whether a rendering pool is needed.
6. **ClickHouse insert batching and merge lag at 3,858 rows/s**, and the point-lookup p99 under concurrent inserts (DR-13, DR-16). Decides whether a KV tier returns.
7. **Frontier refill latency at 1M hosts on a 1.2 TB table** (DR-15), and the re-release path after a forced Redis failover.
8. **A fair columnar-vs-row comparison on target hardware**: Postgres with a covering index (index-only scan), lz4 TOAST, planner free to choose a sequential scan, against ClickHouse on the same rows. The storage difference is measured; the query-time difference is not, and should be before anyone quotes a multiple.